Convolutional Neural Networks

George Michailidis

gmichail@ucla.edu

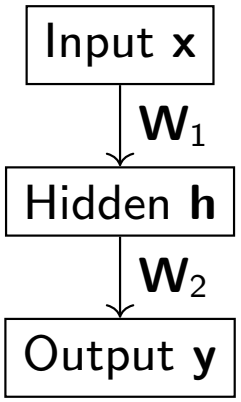
STAT 426

Recap: The Multilayer Perceptron (MLP)

- Definition: A fully connected network consisting of an input layer, one or more hidden layers, and an output layer.
- Mathematical Formulation:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

  Where:
  - $\mathbf{x} \in \mathbb{R}^{d_{in}}$ is the input vector.
  - $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix (dense).
  - $\sigma(\cdot)$ is a non-linear activation (e.g., ReLU, Sigmoid).
- Key Property: Universal Function Approximators.
- Limitation: They are agnostic to the *topology* of the input features.



Data with Topological Structure: Images

Data Representation: The Flattening Operation

- Image Data Structure: An image is intrinsically a 2D grid (or 3D array/tensor) of pixel intensities:

$$I \in \mathbb{R}^{H \times W \times C}$$

  where H is height, W is width, and C is channels.
- The Input Constraint: Standard MLPs require input $\mathbf{x}$ to be a 1D vector.
- The Transformation (Flattening): We must map indices $(i, j, c) \to k$.

$$\mathbf{x} = \text{vec}(I) \in \mathbb{R}^{H \cdot W \cdot C}$$

  where $\text{vec}(\cdot)$ stacks the columns (or rows) of the matrix/tensor in one long column vector
- Immediate Consequence: Geometric relationships are discarded.
  - Pixel (i, j) and Pixel $(i, j+1)$ are neighbors in grid space.
  - In vector $\mathbf{x}$, they may be adjacent, but Pixel (i, j) and Pixel $(i+1, j)$ (vertical neighbor) might be separated by W elements.

The Problem with Flattening: The Permutation Failure

Thought Experiment: Suppose we apply a fixed, random permutation π to the indices of every image in the training set.

$$\mathbf{x}_{permuted} = \pi(\mathbf{x}_{original})$$

- Human Vision: The image becomes visual static (white noise). We cannot recognize the object.
- MLP Vision: The MLP performance will be identical to the non-permuted case (after retraining).
- The Implication:
  - MLPs do not leverage the spatial structure of the data.
  - They treat pixels as independent features, ignoring the strong prior that meaningful information is contained in local neighborhoods.

Motivation: Spatial Variance of Features

- Scenario: Consider an object detection task (e.g., detecting the text "Cat").
- Case A: "Cat" is at the top-left (pixels $x_0 \dots x_{100}$).
- Case B: "Cat" is at the bottom-right (pixels $x_{900} \dots x_{1000}$).

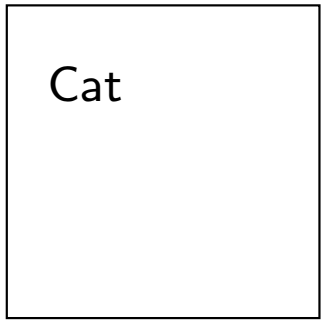


Image A

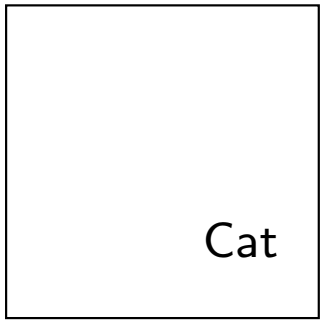


Image B

- To an MLP, Image A and Image B are completely different input vectors (two completely different observations). The active indices are disjoint.
- To a human, these are very simple images containing the word "Cat" in two different spatial locations.

The Failure of Learned Weights in MLPs

- Dense Connections: Every input unit i (i.e., predictor variable i) connects to every hidden unit j via weight $W(i, j)$.
- Lack of Translation Invariance:
  - A hidden unit/neuron learns weights to detect the word "Cat" at pixel coordinates, say $(10 - 20, 10 - 20)$.
  - If the word "Cat" moves to $(200 - 210, 200 - 210)$, the weights associated with indices around $(10 - 20, 10 - 20)$ are irrelevant.
  - The network must re-learn the "Cat" pattern for every possible location in the image grid.
- Inefficiency: We are forced to learn the same feature detector redundant times across the spatial domain.

Consequence: Parameter Explosion

Let us calculate the cost of a Fully Connected (FC) layer for a modest resolution image.

- Input Image: 200×200 pixels, RGB (3 channels).

$$d_{in} = 200 \times 200 \times 3 = 120,000 \text{ inputs}$$

- First Hidden Layer: Suppose we want 1,000 hidden units (a modest number).
- Weight Matrix Calculation:

$$N_{params} = (d_{in} \times d_{hidden}) + d_{hidden}$$

$$N_{params} \approx 120,000 \times 1,000 = 120,000,000 \text{ weights}$$

- Consequences:
  1. Overfitting: With 10^8 parameters in just the first layer, you need a massive data set to prevent memorization of the data and enable generalization.
  2. Memory/Compute: Very expensive for training deep networks on GPUs.

Key Concept: Spatial Invariance

- The Desired Property: The identity (class label) of an object (e.g., image) is invariant to translation (shifting).
- Mathematical Definition: Let $f : \mathcal{X} \to \mathcal{Y}$ be our classification function. Let $T_v(\mathbf{x})$ be a translation operator shifting image $\mathbf{x}$ by vector v.
  We desire:
  $$f(T_v(\mathbf{x})) = f(\mathbf{x})$$
- MLP Reality: Unless trained with data augmentation covering all v for all objects, $f_{MLP}(T_v(\mathbf{x})) \neq f_{MLP}(\mathbf{x})$.

The Solution: Translation Invariance (Scanning)

- Principle 1: Translation Invariance
  - Intuition: A shift in the input **x** should lead to an equivalent shift in the hidden representation **h**, without changing the nature of the feature detected.
- Mathematical Consequence: Weight Sharing
  - In a standard MLP, weights $W_{i,j}$ depend on the specific location (i, j).
  - To satisfy translation invariance, we require W to be independent of location:

$$W_{i,j} \to W_{shared}$$

  - This simplifies the operation to a discrete Convolution:

$$h_{i,j} = \sum_{a,b} V_{a,b} \cdot x_{i+a,j+b}$$

- Result: Instead of learning separate detectors for every location, we learn one detector (V) and "scan" it across the image.

The Principle of Locality & Inductive Bias

- Principle 2: Locality
  - We should not look far from location (i, j) to assess local features.
  - Constraint: We force weights to be 0 outside a small range Δ:

$$|a|, |b| > \Delta \implies V_{a,b} = 0$$

- The "Inductive Bias" Trade-off:
  - The Gain: Massive parameter reduction. *"From millions of parameters to just a few hundred."*
  - The Assumption: We assume meaningful features are local.
  - The Catch: If this bias disagrees with reality (e.g., if longer-range dependencies matter immediately), the model struggles.
- **Note:** We recover global context later by interleaving multiple layers (growing the receptive field).

The Solution: Scanning (Windowing)

- Inspiration: Instead of processing the whole image at once ($\mathbb{R}^{HW} \to \mathbb{R}^d$), process small overlapping patches.
- Mechanism:
  - Define a small feature detector (e.g., 5×5 pixels).
  - Slide this detector over the image.
  - Record the activation at each position.
- Result: This creates a map of where features occur, rather than a single global decision.

Remarks on the Principle of Locality

- Statistical Prior of Images:
  - The correlation between two pixels $\mathbf{x}_i$ and $\mathbf{x}_j$ decays rapidly as the spatial distance $||\text{loc}(i) - \text{loc}(j)||_2$ increases.
  - A pixel at $(0, 0)$ is highly predictive of $(0, 1)$, but statistically independent of $(200, 200)$.
- Sparse Connectivity:
  - MLPs (without significant modifications) enforce Global Connectivity (every input connects to every hidden unit and the same holds for connections between hidden units).
  - We should enforce Local Connectivity (input and hidden units only connect to local neighbors).
- This reduces the parameter space significantly.

Summary: Why we abandon MLPs behind

To process high-dimensional image data effectively, we need an architecture that respects:

1. Structure: Preserves the 2D grid topology (no flattening).
2. Parameter Efficiency: Solves the parameter explosion.
3. Stationarity: Assumes statistics are similar at different locations (if a vertical edge is useful at the top left, it's useful at the bottom right).
4. Locality: Exploits local correlations rather than global dependencies.

Constructing Convolutional Neural Networks

Convolution in Mathematics

The discrete convolution of two signals (e.g. time series or images) f and g is defined as

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]\, g[n-m]$$

Interpretation

- Combines two signals into a third signal
- Each output value is a weighted sum of nearby input values
- The weights are given by the second signal g

(In practice, signals are finite and the sum runs over valid indices only.)

Convolution in Deep Learning

- A small learnable filter (kernel) is slid across the input
- At each spatial location, a **dot product** is computed
- This is a structured linear operation, analogous to a linear layer
- The same filter is reused at every location (weight sharing)

Important Nomenclature Note

- Deep learning libraries implement cross-correlation:

$$(f \star g)[n] = \sum_m f[m]\, g[n+m]$$

- The kernel is not flipped
- Since kernels are learned, this distinction is inconsequential in practice
- By convention, this is still called a convolutional layer

The Components of a Convolutional Layer

1. Input (I): An image or a feature map from a previous layer. Dimensions: $H \times W$ (ignoring channels for now).
2. Filter / Kernel (K): A small matrix of learnable weights. Dimensions: $k \times k$ (e.g. 3×3, 5×5).
3. Output (O): A feature map indicating where and how strongly the filter responds to patterns in the input.

(Real CNNs use multiple filters and multiple channels; we start with one for clarity.)

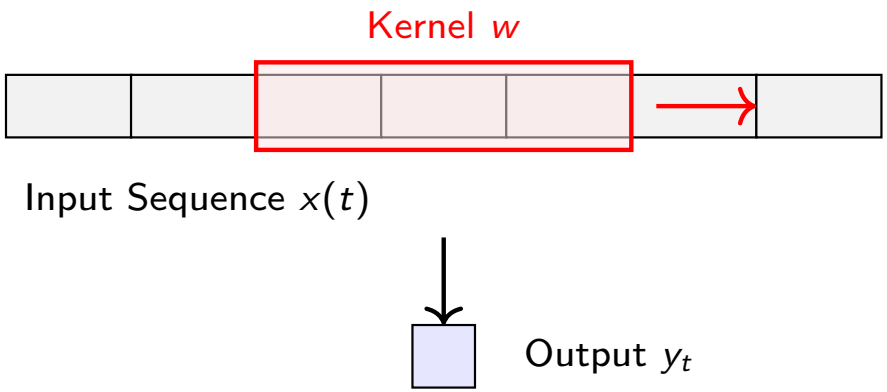
Visualizing 1D Convolution (Temporal Scanning)

Context: Time-series data (Audio, Text, Stock prices).

- Input (x): A sequence of length T.
  Dimension: $\mathbb{R}^T$.
- Kernel (w): A short vector of learnable weights.
  Dimension: $\mathbb{R}^K$ (where $K \ll T$).
- Output (y): The filtered sequence.
  Dimension: $\mathbb{R}^{T-K+1}$.

Math: $y_t = \sum_{k=K}^{K} w_k \cdot x_{t+k}$

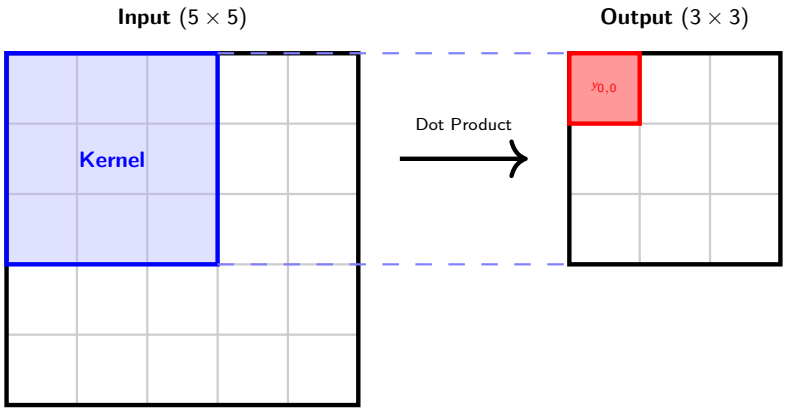
The value y_t is a weighted sum of local values of the signal x in a K-neighborhood around position t.



Visualizing 2D Convolution (Spatial Scanning)

Context: Images or Feature Maps.

- Input (X): A 2D grid of pixels. Dimension: $H \times W$.
- Kernel (W): A 2D feature detector. Dimension: $K \times K$.
- Output (Y): The Activation Map. Dimension: $(H - K + 1) \times (W - K + 1)$.



Mechanism: Element-wise multiplication of the kernel and the image patch, followed by summation.

Step-by-Step Calculation: Illustration of First Two Entries

Matrices:

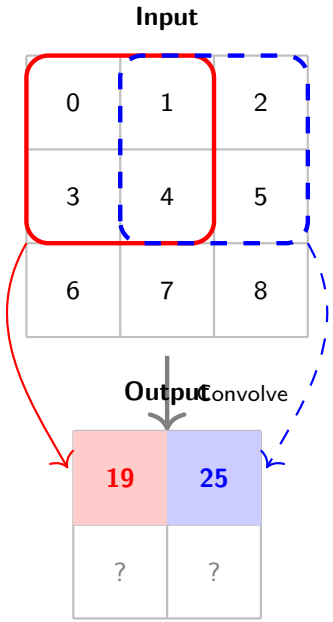
$$\text{Input} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}, \quad \text{Kernel} = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$$

Step 1 (Top-Left): Red Box

$$\begin{aligned} Y_{0,0} &= (0 \cdot 0) + (1 \cdot 1) + (3 \cdot 2) + (4 \cdot 3) \\ &= 0 + 1 + 6 + 12 = \mathbf{19} \end{aligned}$$

Step 2 (Slide Right): Blue Box

$$\begin{aligned} Y_{0,1} &= (1 \cdot 0) + (2 \cdot 1) + (4 \cdot 2) + (5 \cdot 3) \\ &= 0 + 2 + 8 + 15 = \mathbf{25} \end{aligned}$$



Key Concept: Weight Sharing (Parameter Sharing)

The same set of weights (the kernel) is applied at every spatial position (i, j) of the input.

Contrasting MLP and CNN Approach

- **MLP Approach:** A hidden unit "looking" at pixel $(10, 10)$ has different weights than a hidden unit "looking" at pixel $(50, 50)$.
- **CNN Approach:** If a feature (e.g., vertical edge) is useful at $(10, 10)$, it is also useful at $(50, 50)$. The detector is shared
- **Benefit:**
  1. Drastic reduction in parameter count.
  2. Translation Equivariance (detects the object regardless of position).

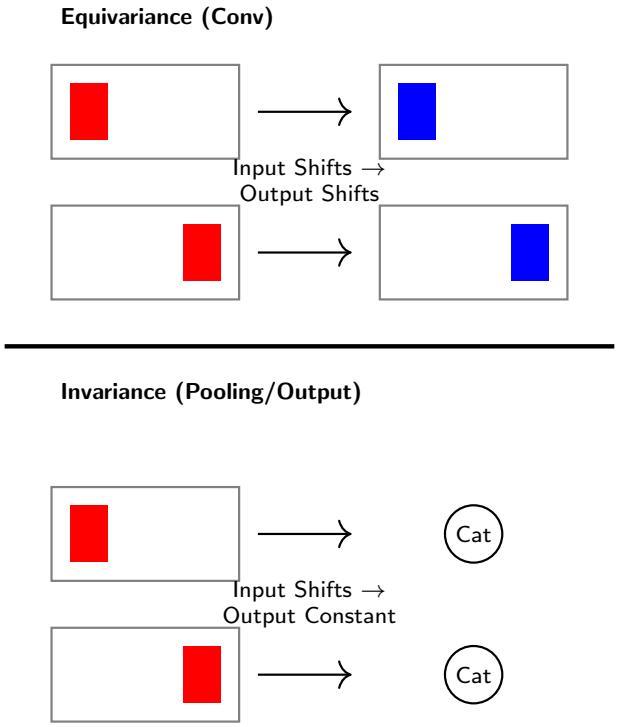
Distinction: Equivariance vs. Invariance

1. Translation Equivariance

- **Definition:** If the input changes, the output changes in the *exact same way*.
- **Math:** $f(shift(x)) = shift(f(x))$
- **Source:** The Convolutional Layer.
- *Intuition:* If the object moves to the right in the image, the "object detection" activation moves to the right in the feature map.

2. Translation Invariance

- **Definition:** If the input changes, the output *remains the same*.
- **Math:** $f(shift(x)) = f(x)$
- **Source:** Pooling & Fully Connected layers.
- *Intuition:* Whether the cat is in the left or right corner, the final prediction is just "Cat."



Imposing Locality via Sparse Connectivity

- Fully Connected Layers (MLPs): Each output unit h_i depends on all input units x_j. The weight matrix W is dense, resulting in many parameters.
- Convolutional Layers: Each output unit h_i depends only on a local neighborhood of the input. This is achieved using small kernels and results in sparse connectivity.

Key idea: CNNs explicitly encode the assumption that nearby inputs are more strongly related than distant ones.

Intuition: The Filter as a Template Matcher

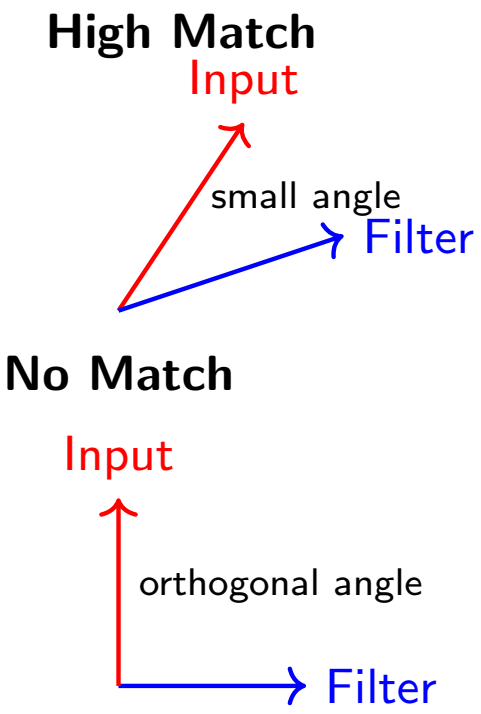
- Analogy: Think of a filter as a "Template."
- Mathematical Basis: Convolution is a sliding dot product.

$$\text{Output} = \mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$$

- Geometric Interpretation:

$$\mathbf{w} \cdot \mathbf{x} = ||\mathbf{w}||||\mathbf{x}|| \cos(\text{angle})$$

  - Max Activation: When angle ≈ 0 (input looks exactly like the filter).
  - Zero Activation: When angle $= 90^\circ$ (input is uncorrelated).
  - Negative Activation: When angle $= 180^\circ$ (input is the inverse).

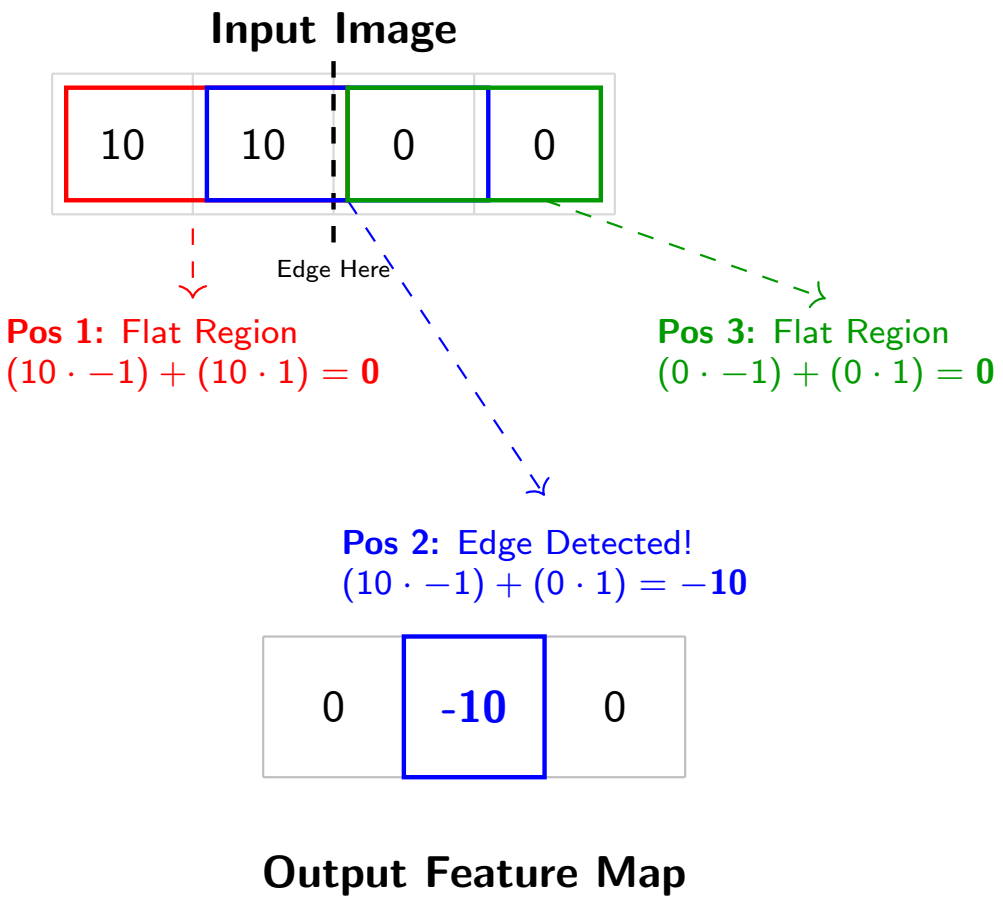


Concrete Example: Detecting a Vertical Edge

The Setup:

1. Kernel: A simple difference detector $[-1, 1]$.
2. **Image:** A bright region (10) transitioning to a dark region (0).

Goal: The output should flag exactly where the value changes.



From Feature Engineering to End-to-End Learning

- Old School Computer Vision (Pre-2012): Data scientists manually calculated filter values (e.g., Sobel, Canny, Gabor filters) to extract edges or textures.
- Deep Learning Approach:
  - We initialize filters randomly (e.g., Gaussian noise).
  - We treat the filter values as parameters θ.
  - We use Backpropagation to update θ based on the training loss function.
- Outcome: The deep network learns whatever features (edges, colors, blobs) minimize the training loss function.

The Bias Term

- As with a linear layer $y = \mathbf{wx} + b$, a convolution implements an affine (linear)transformation: a linear operation followed by the addition of a bias term.
- Equation:

$$\mathbf{Y} = \mathbf{X} * \mathbf{W} + b$$

- Broadcasting the Bias:
  - Each filter has a single scalar bias parameter.
  - This bias is added to *every spatial location* of the corresponding output feature map.
- Interpretation: The bias allows a filter to activate even when the input values are zero or weak.

Convolutions in Different Dimensions

The convolution operation generalizes to N dimensions:

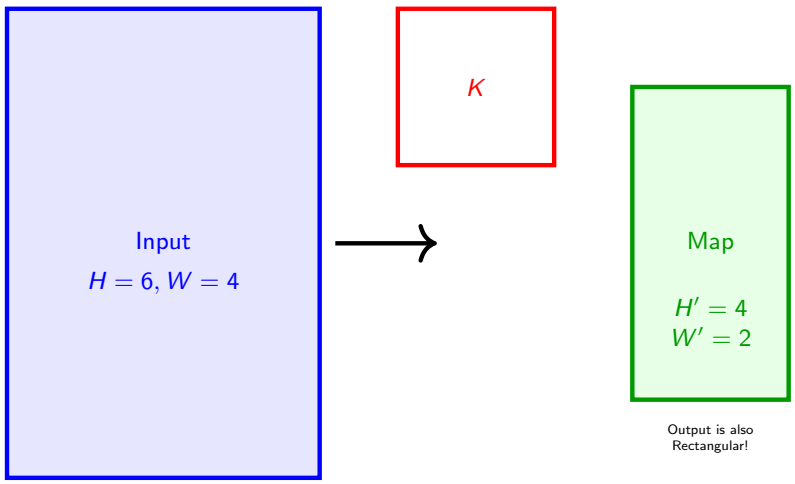
- 1D Conv: **Text, Audio, Time-Series.** Input shape (N, L, C). Sliding over time L.
- 2D Conv: **Images.** Input shape (N, H, W, C). Sliding over spatial H, W. *(focus of this deck)*
- 3D Conv: **Video, Volumetric Data (MRI).** Input shape (N, H, W, D, C). Sliding over Height, Width and Depth.

Feature Maps (Activation Maps)

- Definition: The 2D grid output produced by convolving a single filter over the input.
- Interpretation: A map of "evidence." High value at location (i, j) means the pattern is present at (i, j).
- Spatial Dimensions of the Feature Map (assuming "valid convolution") discussed next

Generalizing Dimensions: Beyond Squares

1. Inputs are Rectangles ($H \times W$)
   - Real images are rarely perfect squares (e.g., 1920×1080).
   - Distinguish between:
     - **Height (H):** The number of rows.
     - **Width (W):** The number of columns.
   - The kernel slides vertically H times and horizontally W times.
2. Output Size Calculations separately for height and width



A Note on Kernel Dimensions

- Kernels are typically Square ($K \times K$)?
  - We typically want to detect features (corners, blobs) symmetrically in all directions.
  - A square 3×3 or 5×5 is a good approximation of a circular "field of view."
- Rectangular Kernels ($K_h \times K_w$)
  - In advanced architectures or specific domains (e.g., audio spectrograms), we might use a 1×7 kernel to capture long horizontal patterns.

The General Case Formula

If we have a rectangular input (H_{in}, W_{in}) and a rectangular kernel (K_h, K_w):

$$\text{Output Height} = H_{in} - K_h + 1$$

$$\text{Output Width} = W_{in} - K_w + 1$$

Summary: The Convolution Operation

1. Scanning Window: We slide a filter over the spatial dimensions of the input.
2. Parameter Sharing: The same weights are used everywhere, enforcing translation invariance.
3. Locality: Output neurons only connect to a local patch of inputs.
4. Feature Mapping: The result is a map indicating where specific local patterns (edges, blobs) occur in the image.

The Shrinking Image Problem

Convolution reduces spatial resolution when no padding is used.

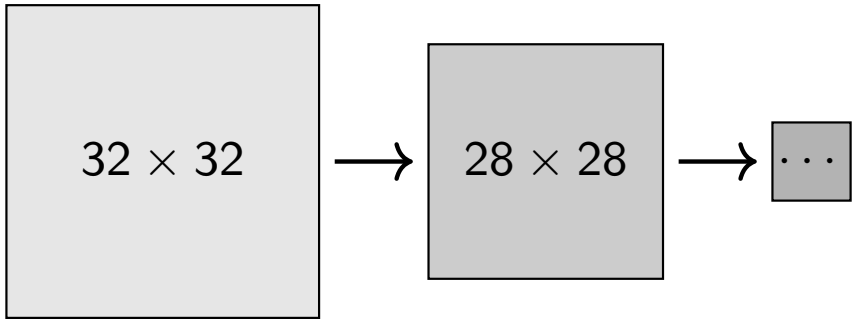
If the input is $N \times N$ and the kernel is $K \times K$, then

$$\text{Output size} = (N - K + 1) \times (N - K + 1)$$

- Example: Input 32×32, kernel 5×5

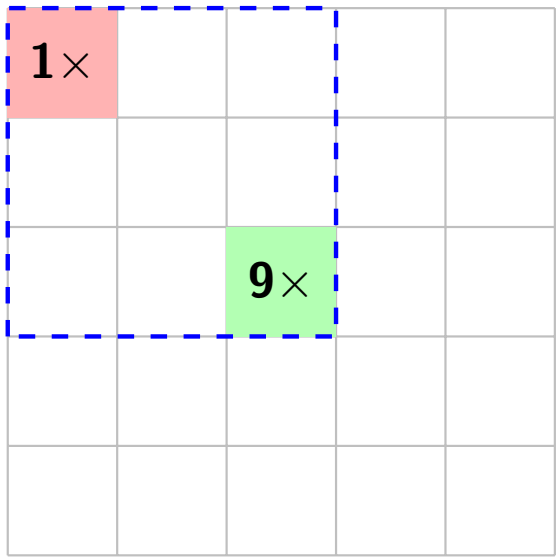
$$32 - 5 + 1 = 28 \;\Rightarrow\; 28 \times 28$$

- Why this is a problem:
  - Stacking many convolutional layers rapidly shrinks the feature maps
  - After only a few layers, spatial information is lost
  - This limits how deep the network can be



The Boundary Problem

- Uneven Contribution of Pixels:
  - A pixel near the center of the image is covered by the kernel many times as it slides across the input.
  - A pixel near the boundary is covered far fewer times, and corner pixels may be used only once.
- Consequences:
  1. Information near image boundaries is underrepresented.
  2. The network implicitly places more emphasis on central regions.



Number of times a 3×3 kernel includes a pixel.

Solution: Padding

- Definition: Padding refers to artificially extending the input by adding extra rows and columns around its boundary before applying convolution.
- Common Types:
  - **Zero Padding:** Pad with zeros (most common in deep learning).
  - **Reflection Padding:** Mirror the input at the boundary.
- Goals of Padding:
  1. Control the spatial resolution of the output (often to preserve it).
  2. Ensure boundary pixels are treated similarly to interior pixels.

Padding allows convolutional layers to be stacked without losing spatial information too quickly.

Visualizing Zero Padding

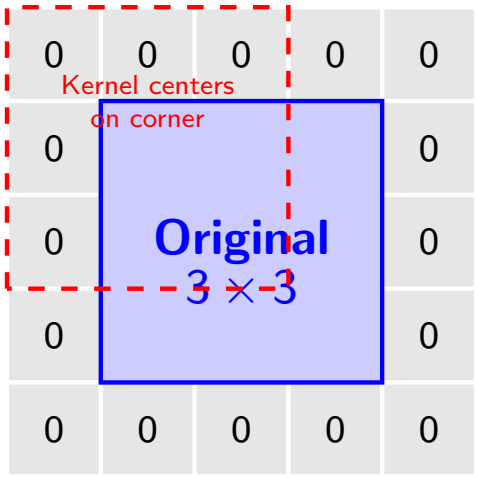
Scenario:

- Input: 3×3 (Blue)
- Padding: $P = 1$ (Gray)
- Effective Input: 5×5
- Kernel: 3×3

Effect: Apply 3×3 kernel to 5×5 input:

$$5 - 3 + 1 = 3$$

Output is 3×3 (same as original)



Standard Terminology: *Valid* vs. *Same*

Valid Convolution

- No padding is applied ($P = 0$).
- The filter is only applied where it fully overlaps the input.
- **Result:** Spatial dimensions shrink after convolution.

Same Convolution

- Padding is chosen so that the output has the same spatial size as the input.
- This simplifies architecture design by keeping width and height constant while increasing feature depth.
- Padding rule (stride $= 1$):

$$P = \frac{K - 1}{2}$$

- *Note:* This requires the kernel size K to be odd (e.g. $3, 5, 7$).

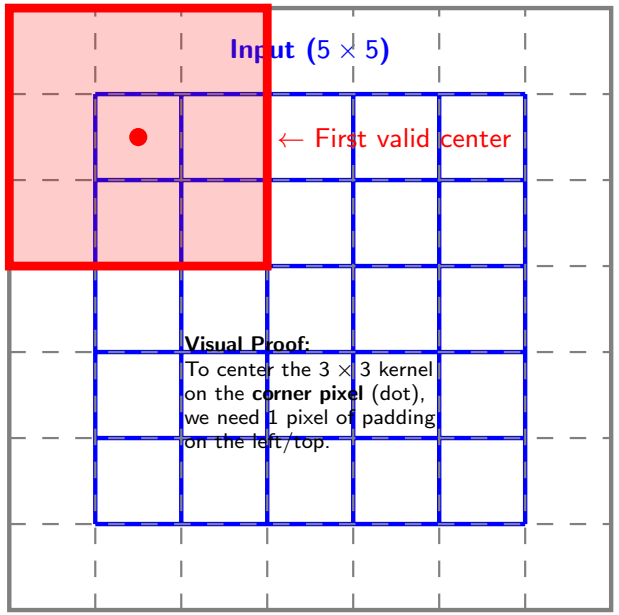
Elaborating on "Same Convolution"

We often want the **Output Size** to exactly match the **Input Size**.

- Input: $H \times W \rightarrow$ Output: $H \times W$.
- This allows us to stack many layers without the image shrinking away.

The "Stride = 1" Requirement

- Stride is the step size of the filter.
- To keep the same resolution, the filter must step 1 pixel at a time ($S = 1$).
- If $S > 1$, the image size would halve (or quarter), regardless of padding.



Padding $P = 1$ adds border

More on Stride: The Step Size

- Definition: The stride (S) specifies how many pixels the filter moves at each step as it slides across the input.
- Standard Convolution ($S = 1$):
  - The filter moves one pixel at a time.
  - Produces a high-resolution output feature map.
- Strided Convolution ($S > 1$):
  - Example: $S = 2$ (the filter moves two pixels per step).
  - Some spatial locations are skipped.
  - **Effect:** Downsampling — the output has smaller spatial dimensions.

Key idea: Stride trades spatial resolution for computational efficiency.

Visualizing Stride: S=1 vs S=2

Setup:

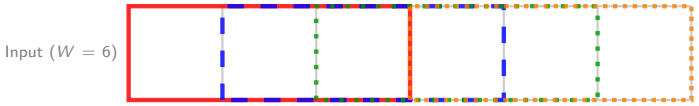
- **Input Width (W):** 6 units
- **Kernel Size (K):** 3 units

Output Calculation

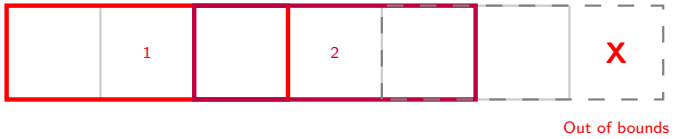
$$O = \lfloor \frac{W - K}{S} \rfloor + 1$$

- **Stride 1:** $\frac{6-3}{1} + 1 = \mathbf{4}$ outputs.
- **Stride 2:** $\lfloor \frac{6-3}{2} \rfloor + 1 = \lfloor 1.5 \rfloor + 1 = \mathbf{2}$ outputs.

Scenario A: Stride $S = 1$ (4 Outputs)



Scenario B: Stride $S = 2$ (2 Outputs)



The Master Formula

To compute the output size along a single spatial dimension (height or width):

$$\boxed{O = \left\lfloor \frac{W - K + 2P}{S} + 1 \right\rfloor}$$

Variables:

- W: Input size (height or width)
- K: Kernel size
- P: Padding
- S: Stride

Details:

- $\lfloor \cdot \rfloor$: floor (round down)
- If $(W - K + 2P)$ is not divisible by S, extra pixels are ignored
- Essential for avoiding shape mismatch errors

Calculation Practice Problem 1: Standard Shrinkage

Problem

- **Input:** 32×32 image ($W = 32$)
- **Filter:** 5×5 ($K = 5$)
- **Padding:** None ($P = 0$)
- **Stride:** Standard ($S = 1$)

Calculation:

$$O = \frac{32 - 5 + (2 \times 0)}{1} + 1$$

$$O = \frac{27}{1} + 1 = \mathbf{28}$$

Result: Output feature map is $\mathbf{28} \times \mathbf{28}$.

Calculation Practice Problem 2: Preserving Size

Problem

- **Input:** 32×32 image ($W = 32$)
- **Filter:** 5×5 ($K = 5$)
- **Padding:** 2 ($P = 2$)
- **Stride:** Standard ($S = 1$)

Calculation:

$$O = \frac{32 - 5 + (2 \times 2)}{1} + 1$$

$$O = \frac{32 - 5 + 4}{1} + 1$$

$$O = 31 + 1 = \mathbf{32}$$

Result: Output feature map is $\mathbf{32} \times \mathbf{32}$. (Size Preserved!)

Calculation Practice Problem 3: Downsampling

Problem

- **Input:** 32×32 image ($W = 32$)
- **Filter:** 2×2 ($K = 2$)
- **Padding:** None ($P = 0$)
- **Stride:** 2 ($S = 2$)

Calculation:

$$O = \frac{32 - 2 + (2 \times 0)}{2} + 1$$

$$O = \frac{30}{2} + 1$$

$$O = 15 + 1 = \mathbf{16}$$

Result: Output feature map is $\mathbf{16} \times \mathbf{16}$. (Dimensions Halved).

Calculation Practice Problem 4: Different Height and Width

Problem

- **Input:** 28×40 image (Height $H = 28$, Width $W = 40$)
- **Filter:** 3×3 ($K = 3$)
- **Padding:** 1 ($P = 1$)
- **Stride:** 2 ($S = 2$)

Height calculation:

$$O_H = \left\lfloor \frac{28 - 3 + 2}{2} + 1 \right\rfloor = \left\lfloor \frac{27}{2} + 1 \right\rfloor = \mathbf{14}$$

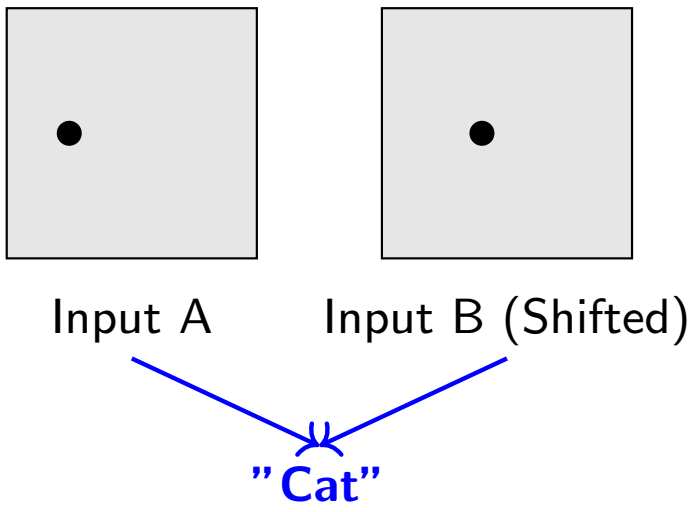
Width calculation:

$$O_W = \left\lfloor \frac{40 - 3 + 2}{2} + 1 \right\rfloor = \left\lfloor \frac{39}{2} + 1 \right\rfloor = \mathbf{20}$$

Result: Output feature map is $\mathbf{14} \times \mathbf{20}$.

The Need for Pooling: Invariance vs. Equivariance

- Convolution is Equivariant: If the input shifts by 1 pixel, the feature map shifts by 1 pixel.
- Rationale: We do not care exactly where the cat's ear is (pixel 100 vs 101). We just care that it is there.



The Objective: Invariance

- We want the network to recognize the object regardless of small translations, distortions, or shifts.
- Solution: Pooling (subsampling).

What is Pooling?

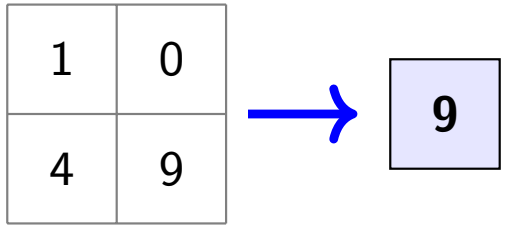
- Definition: A process of replacing a region of the input with a single summary statistic.
- Mechanism:
  - Similar to convolution: We slide a window (e.g., 2×2) across the input.
  - Key Difference: There are no weights (dot products). We perform a fixed deterministic operation.
- Key Property: No parameters to be leaned, it is simply a calculation.

Max Pooling: The Standard

Output the maximum value observed in the current window.

Intuition:

- Think of each pixel in the feature map as a "hidden unit activation."
- Max Pooling asks: *"Did this feature occur anywhere in this region?"*
- We keep the strongest signal and discard the noise.



Average Pooling

Output the arithmetic mean of the values in the current window.

- Intuition: "How strong is this feature on average in this area?"
- Effect:
  - Smooths out the image/feature map.
  - Suppresses noise but also dilutes strong edge features.
- Usage Trend:
  - **Historical:** Used in early CNNs.
  - **Modern:** Rarely used between layers; mostly used at the very end of the network (Global Average Pooling).

Pooling as Downsampling

- Standard Configuration (Non-Overlapping): To reduce dimensions cleanly, we typically set Stride = Filter Size.
  - Filter $2 \times 2 \rightarrow$ Stride 2 (Halves the size).
  - Filter $3 \times 3 \rightarrow$ Stride 3 (Thirds the size).
- Overlapping Pooling (Alternative): You can use Stride $<$ Filter Size (e.g., Filter 3×3, Stride 2). This creates "overlapping windows," which can reduce information loss but is less common today.
- Calculation: Using the standard 2×2 with Stride 2:

$$W_{out} = \lfloor \frac{W_{in} - 2}{2} \rfloor + 1 = \frac{W_{in}}{2}$$

- Data Reduction Impact:
  - Reducing H and W by half reduces the total area by factor of 4.
  - 75% of the activations are discarded.
  - This forces the network to keep only the most salient features.

Visualizing Max Pooling (2x2, Stride 2)

Input (4×4)

1	0	2	3
4	**6**	6	**8**
3	1	1	0
1	0	2	**4**

Max Pool →

Output (2×2)

6	8
3	4

Robustness to Translation (Visual Proof)

Original Signal

0	**9**	0	0

$\downarrow$ Max Pool (2×1) $\downarrow$

9	0

Shifted Signal (Right 1px)

0	0	**9**	0

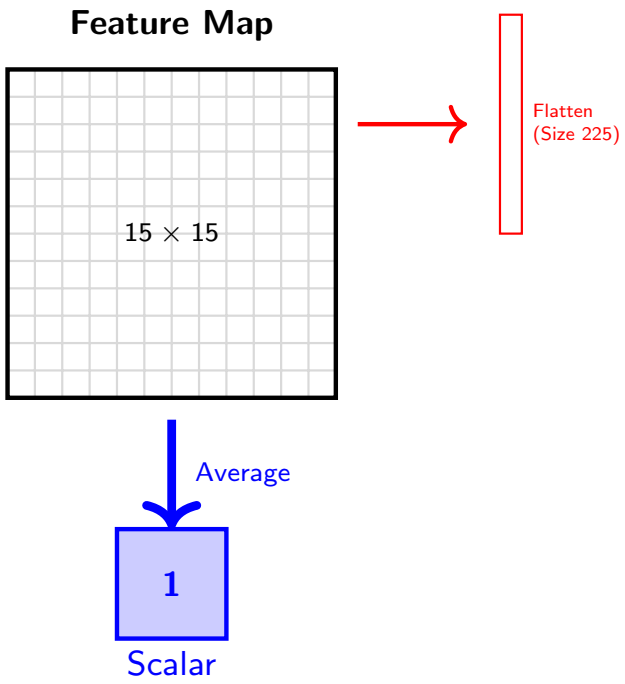
$\downarrow$ Max Pool (2×1) $\downarrow$

0	9

Note: While the *exact* position in the output changed slightly, the strong activation (9) was preserved and not lost. In deeper layers, this allows the network to say "I saw a 9 roughly in this area."

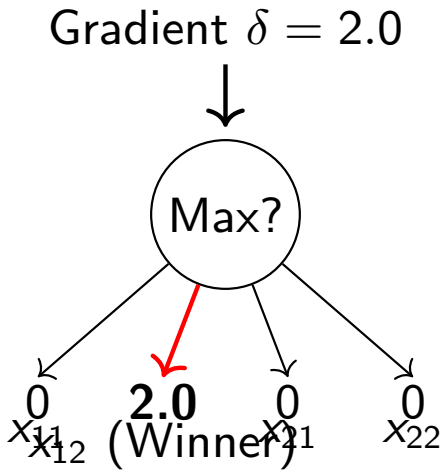
Global Average Pooling (GAP)

- Context: At the end of the network, suppose we have a feature map of size 15×15.
- Old Way (Flatten):
  - We unroll the 15×15 grid into a long vector of size **225**.
  - This vector connects to the final Dense layer.
  - *Downside:* Requires defining weights for all 225 inputs.
- Modern Way (GAP):
  - We take the **Average** of the entire 15×15 map.
  - Input: 15×15 grid $\rightarrow$ Output: 1 single value
- Benefit:
  - Extreme Compression: Reduces the input to the classifier from 225 dimensions to just 1.
  - Prevents Overfitting: Eliminates tons of parameters in the dense layers.



Backpropagation: Routing the Gradient

How do we pass loss function derivatives through a function with no weights?



Max Pool Backward:
The gradient flows only to the pixel that was the Max.

- If x_1 was the winner, $dL/dx_1 = dL/dy$.
- Others get 0.

Average Pool Backward:
The gradient is split equally $(1/W)$ among all input pixels.

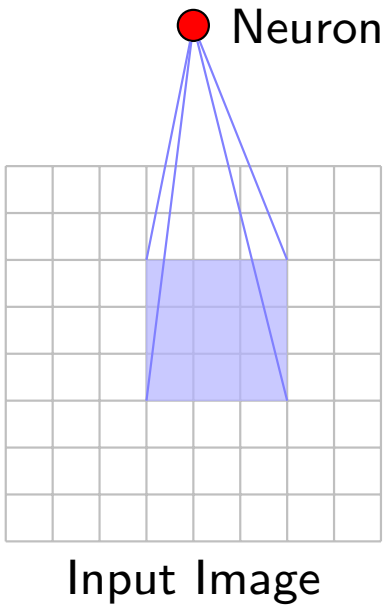
Summary: Pooling Layers

1. Dimensionality Reduction: The primary tool for managing computational cost and memory. (e.g., $224 \to 112 \to 56 \ldots$).
2. Invariance: Provides robustness to small shifts and distortions in the input image.
3. No Parameters: Pooling adds zero learnable weights to the network, reducing the risk of overfitting.
4. Standard Practice: Max Pooling (2×2, Stride 2) is the industry standard for intermediate layers.

Receptive Fields: The "Field of View"

The Receptive Field (RF) of a neuron is the specific region of the original input image that influences that neuron's activation.

- Analogy: If you look through a straw, you only see a small circle of the world. That circle is your current receptive field.
- **Layer 1:** The RF is simply the Kernel Size (e.g., 3×3).
- Challenge: How does a neuron know about the *whole* image if it only looks at 3×3 pixels?

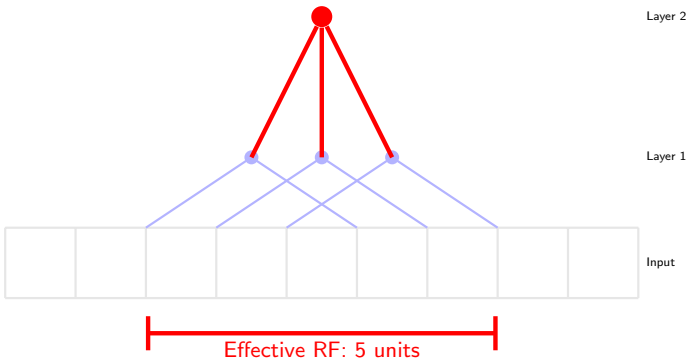


Growing Receptive Fields

As we go deeper, each hidden unit can incorporate information from a larger region of the input.

The Expansion Effect

- **Layer 1:** Each hidden unit sees a 3×3 patch of the input.
- **Layer 2:** Each nhidden unit sees a 3×3 patch of Layer 1.
- **Result:** A Layer 2 hidden unit indirectly sees a 5×5 patch of the input.

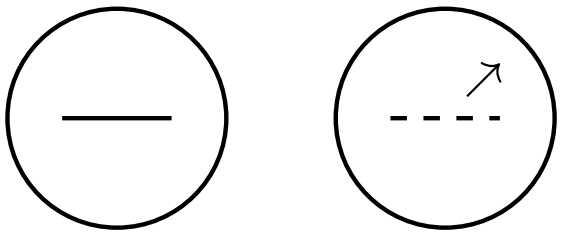


Rule of thumb (for 3×3 kernels, stride $= 1$):

$$\text{Receptive field size} \approx 1 + 2 \times (\text{depth})$$

Biological Inspiration: Hubel & Wiesel (1959)

- They recorded neurons in a cat's visual cortex while showing it slides of light bars.
- 1. Simple Cells: Activate only when an edge is at a specific *orientation* and specific *location*.
  2. Complex Cells: Activate if the edge is present anywhere in a region (Motion/Invariance).
- The Deep Learning Parallel:
  - **Convolution** $\approx$ Simple Cells (Feature detection).
  - **Pooling** $\approx$ Complex Cells (Spatial invariance).



Simple Cell (Orientation specific) Complex Cell (Movement/Shift invariant)

The Predecessor: Neocognitron (Fukushima, 1980)

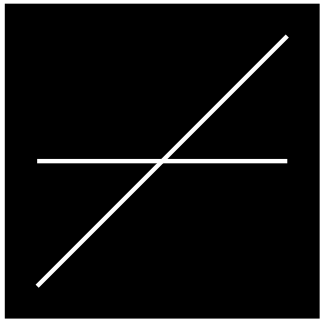
- Concept: The first architecture to explicitly arrange layers in a cascade:

$$S \to C \to S \to C$$

- Structure:
  - **S-Layers (Simple):** Feature extraction (Convolution).
  - **C-Layers (Complex):** Pooling/Blurring for invariance.
- However, it did not use Backpropagation. It relied on unsupervised "winner-take-all" learning (Hebbian), which made it hard to train for complex tasks.

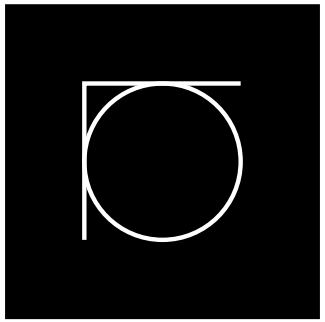
What do the filters actually learn?

Layer 1



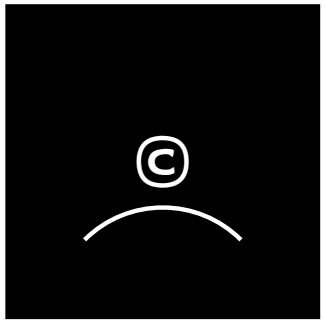
Oriented Edges,
Gradients, Colors.

Layer 2



Textures, Corners,
Circles, Spirals.

Layer 3+



Parts of objects:
Eyes, Text, Wheels.

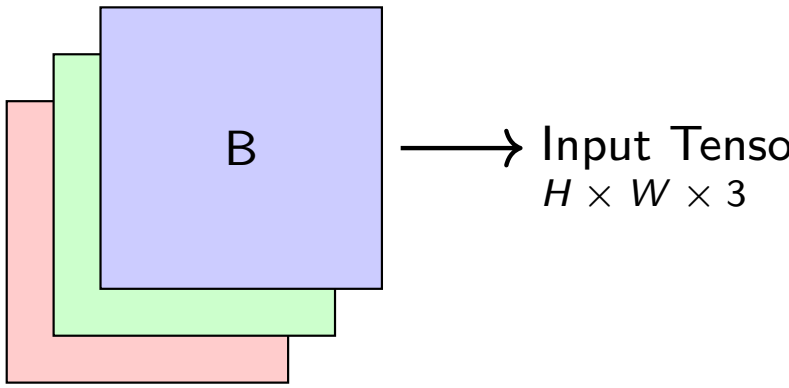
Crucial Note: We do not program these. The network *learns* this hierarchy automatically to minimize the loss.

Summary: Receptive Fields Hierarchies

1. Hierarchical Construction: CNNs build complex understandings from simple blocks (Edges $\rightarrow$ Shapes $\rightarrow$ Objects).
2. Receptive Fields: Hidden units in deeper layers "see" larger portions of the input image, allowing for context awareness.
3. Biological Roots: The S-Cell/C-Cell model from Hubel & Wiesel directly inspired the Convolution/Pooling architecture.

Beyond Grayscale: Multiple Input Channels

- So far, we treated inputs as 2D matrices ($H \times W$). However, real digital images are 3D tensors: $H \times W \times C_{in}$.
  - **RGB Images:** 3 Channels (Red, Green, Blue).
  - **Satellite Imagery:** Often 10+ spectral channels.
  - **Hidden Layers:** A layer deep in the network might receive 256 channels from the previous layer.
- Key Question: How do we apply a filter to a 3D volume?



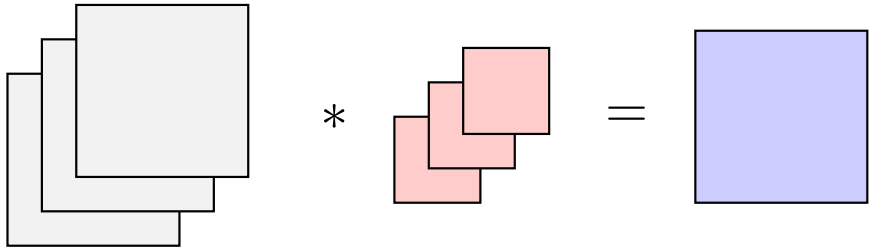
The Mechanics of Multi-Input Convolution

Rule: Depth Matching

If the input has C_{in} channels, the Kernel must also have C_{in} channels. The Kernel is a 3D tensor: $k_h \times k_w \times C_{in}$.

Mechanics:

1. Align the C_{in} kernel channels with the C_{in} input channels.
2. Perform 2D cross-correlation on each channel independently.
3. Sum the results element-wise to get a single 2D scalar map.



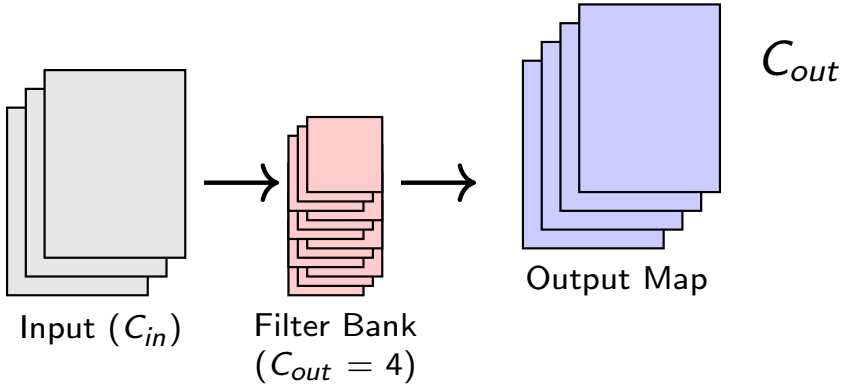
Input ($C_{in} = 3$) Kernel ($C_{in} = 3$) Output (1 Channel)

$$\text{Output}(i,j) = \sum_{c=1}^{C_{in}} (\text{Input}_c * \text{Kernel}_c)(i,j)$$

Multiple Output Channels: I

We need to extract many different types of information from the image simultaneously.

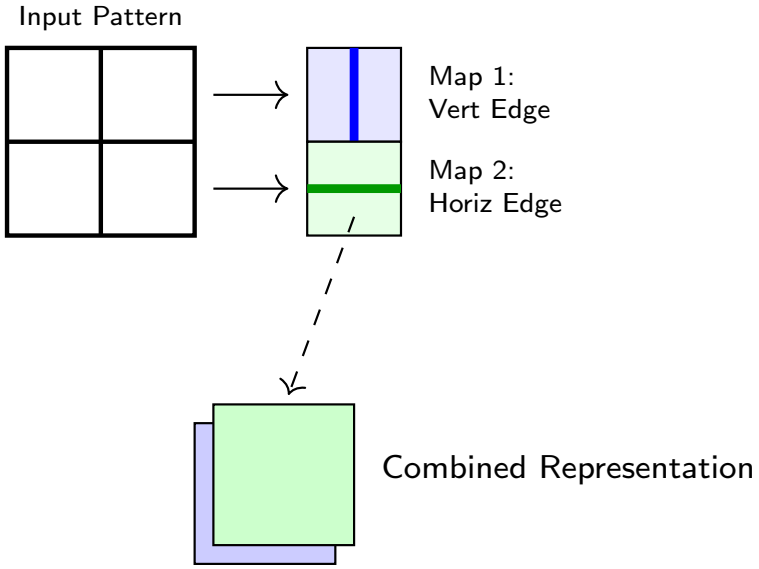
- We create a Filter Bank of C_{out} distinct kernels. Each kernel learns to recognize a different pattern.
- Dimensions:
  - **Input:** $H \times W \times C_{in}$
  - **Kernel:** $k \times k \times C_{in}$
  - **Weights:** $C_{out} \times (k \times k \times C_{in})$
  - **Output:** $H \times W \times C_{out}$



Multiple Output Channels - II

One Filter = One Pattern A single convolution filter (e.g., 3×3) is limited. If its weights are set to detect Vertical Edges, it is mathematically incapable of detecting Horizontal Edges.

- To understand an image, we need a team of "specialists":
  - **Filter 1:** Looks for Vertical Lines.
  - **Filter 2:** Looks for Horizontal Lines.
  - **Filter 3:** Looks for Color Gradients.
- Stacking all filters: When we stack these outputs ($H \times W \times C_{out}$), the vector at pixel (i, j) becomes a Feature Descriptor describing exactly "what" is at that location.



The 1×1 convolution - I

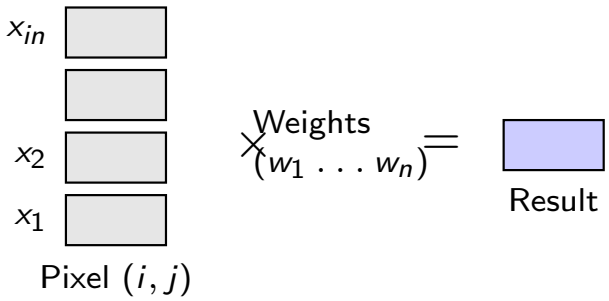
Focus on a single pixel location (i, j).

- There are C_{in} values (the "channels").
- The 1×1 Kernel is just a list of weights matching those channels.

The Operation of the 1×1 convolution: Multiply each channel value by its weight and sum them up.

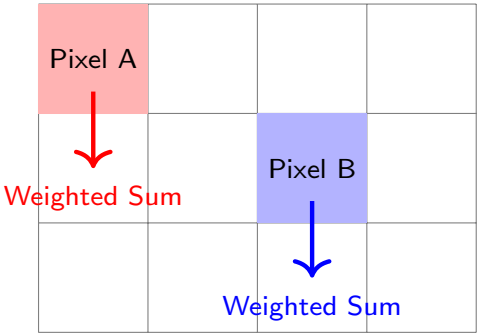
$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n$$

End result: Obtain a single number for that pixel, summarizing the information across $C_i n$ channels.



Global view of 1×1 convolution - Applying it to the Image

Now, we apply that same set of weights to every pixel.



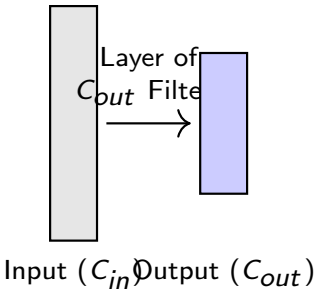
Crucial Point:
Pixel A does NOT talk to Pixel B.

There is **no spatial mixing**.
We only mix the **colors/channels** *inside* each pixel.

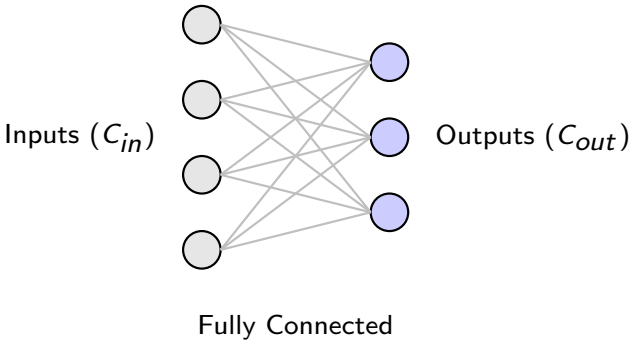
The MLP View

If we ignore the spatial grid (image) and just look at the channel vectors, the math is identical.

View A: The "1x1 Conv Layer"



View B: The "MLP View"



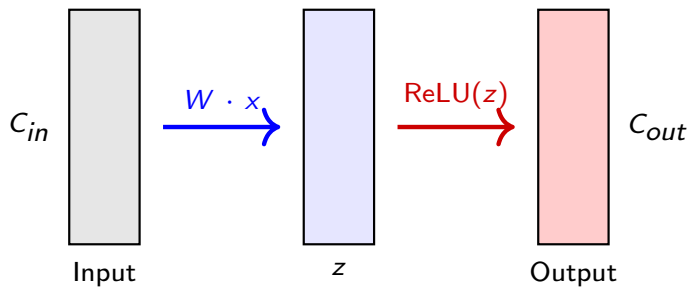
The Mapping

- **Input Nodes** = Input Channels (C_{in})
- **Output Nodes** = Number of Filters (C_{out})
- **Result:** It is a standard Dense Layer applied to every pixel.

Completing the "Hidden Unit"

The 1×1 Convolution provides the Linear part of a hidden unit. To make it a powerful feature extractor, we immediately add a Non-Linear activation.

1. The Linear Part
(The 1×1 Conv) (The Activation)



2. The Non-Linear Part

The "1×1 Layer" in code usually includes both.

Without ReLU:

- The network remains linear.
- Two 1×1 layers stacked = One single linear matrix.

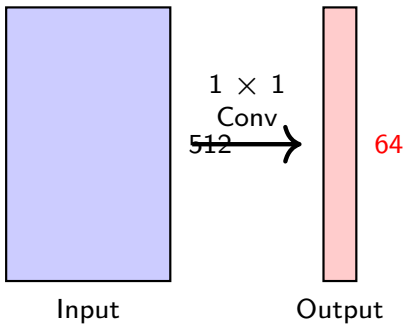
With ReLU:

- The network becomes "Deep".
- We can learn complex decision boundaries between channels.

Summary: Why use 1×1 Convolutions?

1. The Bottleneck (Efficiency):
   - Running a 3×3 filter on 512 channels is extremely expensive.
   - We use a 1×1 layer to compress channels (e.g., $512 \rightarrow 64$) *before* the heavy processing.
2. Feature Fusion (The MLP Effect):
   - It acts as a mixer. It combines independent features (e.g., "Red" + "Vertical") into a new, composite feature.
3. Deepening without Shrinking:
   - We can add more layers (more ReLUs/Non-linearity) to increase model capacity without reducing the image's spatial resolution ($H \times W$).

The Bottleneck Visualized



Drastic reduction in computation cost.

How Feature Fusion Works

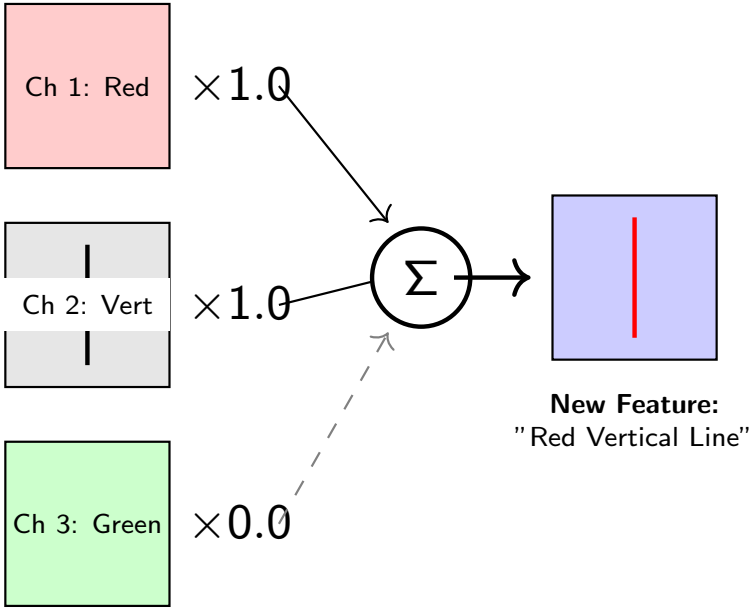
The Mechanism: Linear Combination

The 1×1 kernel is a set of learned weights $(w_1, w_2, \dots)$.

- Scenario: We want to detect a "Red Vertical Edge".
- Input Channels:
  - Ch 1: Detects Red colors.
  - Ch 2: Detects **Vertical lines**.
  - Ch 3: Detects Green colors.

The Weights act as Selectors: To fuse Red + Vertical, the network learns:

$$w = [\mathbf{1.0}, \mathbf{1.0}, \mathbf{0.0}]$$



$$\text{Output} = 1 \cdot \text{Red} + 1 \cdot \text{Vert} + 0 \cdot \text{Green}$$

Parameter Count Summary

Setup: We define a standard layer with kernel size $K \times K$, input channels C_{in}, output channels C_{out}, and output spatial size $H \times W$.

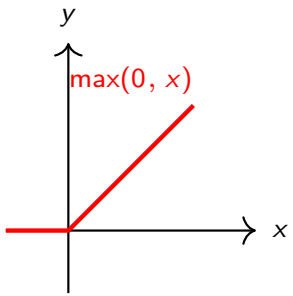
1. Convolution Cost (Expensive)

Parameter Count (Storage):

$$\text{Params} = (\underbrace{K^2 \times C_{in}}_{\text{Kernel}} + \underbrace{1}_{\text{Bias}}) \times C_{out}$$

Note: 1×1 *Convs reduce* K *to 1, drastically cutting costs.*

2. ReLU Cost (Free)



- **Params: +0**
  (Fixed function)

Conclusion: Model size is determined almost entirely by the **Linear** layers.

The Limits of CNNs: Rotation & Scale

- What CNNs have: Translation Equivariance.
  - If the cat moves left, the activation moves left.
  - Pooling gives us small Translation *Invariance*.
- What CNNs lack: Rotational and Scale Invariance.
  - A standard vertical edge filter $\begin{bmatrix} -1 & 1 \end{bmatrix}$ cannot detect a horizontal edge.
  - If an object rotates 90°, the CNN might fail completely unless it has seen rotated versions during training.

Standard filters are rigid templates

Solution: Data Augmentation

Generating "new" training samples from existing ones by applying random, realistic transformations.

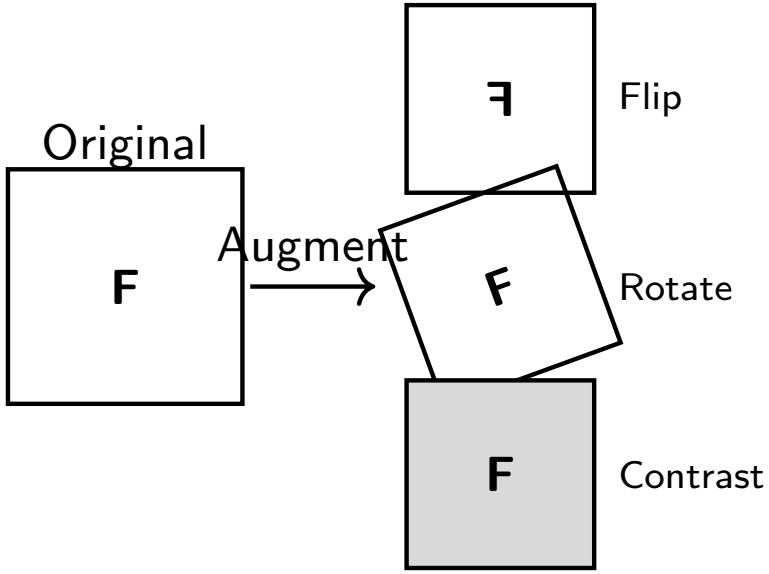
- If we want the network to be invariant to rotation, we must show it rotated images during training.
- Procedure:
  - Training: Load image $x \rightarrow$ Apply random transform $T(x) \rightarrow$ Train.
  - Testing: Usually use the clean, original image.
- This acts as a powerful Regularizer (prevents overfitting).

Common Augmentation Strategies

1. Geometric Transformations
   - Flipping: Horizontal (good for cats), Vertical (bad for digits).
   - Rotation: Randomly rotate $\pm 15^\circ$.
   - Cropping: Zoom into random parts of the image.
   - Translation: Shift pixels slightly.

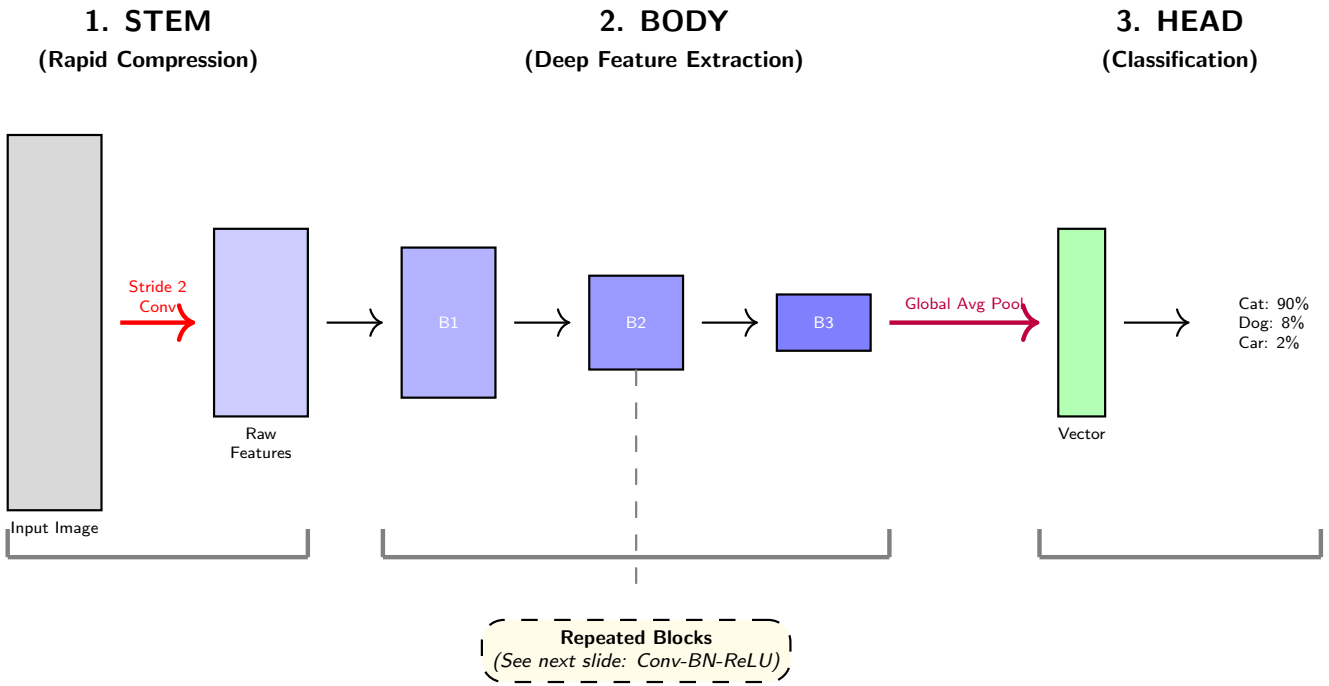
2. Photometric Transformations
   - Color Jitter: Randomly change Brightness, Contrast, Saturation.
   - Noise Injection: Add Gaussian noise.
   - Cutout: Randomly mask out a square region of the image (forces network to look at context).

Visualizing Augmentation



Note: The label ("F") remains the same for all variants!

Visualizing the CNN Pipeline (Macro View)



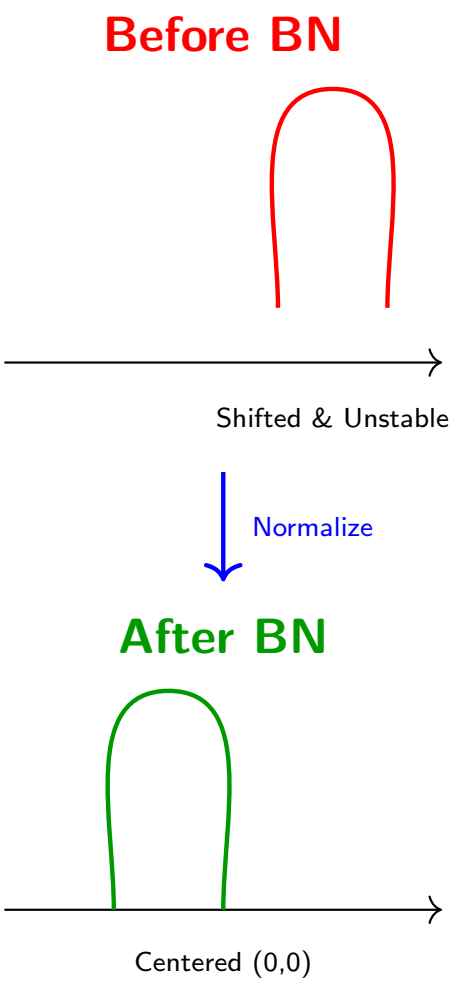
Digression: What is Batch Normalization (BN)?

The Problem of "Covariate Shift"

- As data flows through the network, the values can drift.
- Some become very large (e.g., 1000), some tiny (e.g., 0.001).
- This confuses the network and slows down training.

The Solution: Re-Centering

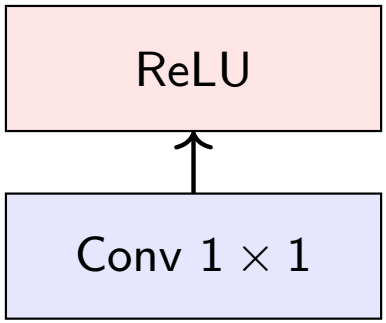
- BN looks at the current batch of images.
- It subtracts the mean (μ) and divides by the variance (σ).
- End Result: The data is forced into a standard range (roughly -1 to 1).



The Conv-BN-ReLU block

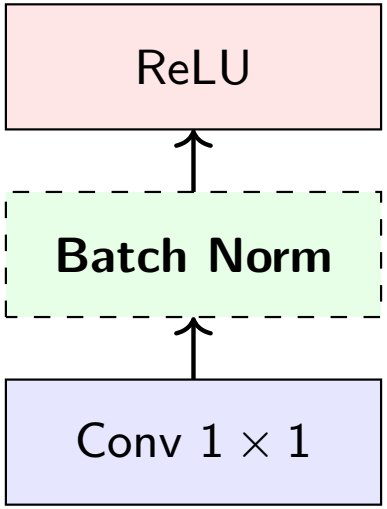
We previously saw the simplified 1×1 convolution and connect it to the practical implementation.

The "Theory" View



"Linear $\rightarrow$ Non-Linear"

The "Practice" View



"Linear $\rightarrow$ Stabilize $\rightarrow$ Non-Linear"

Takeaway message: The ReLU is exactly the same. We just insert Batch Norm in the middle to make training faster and more stable.

Getting to the Class Probabilities

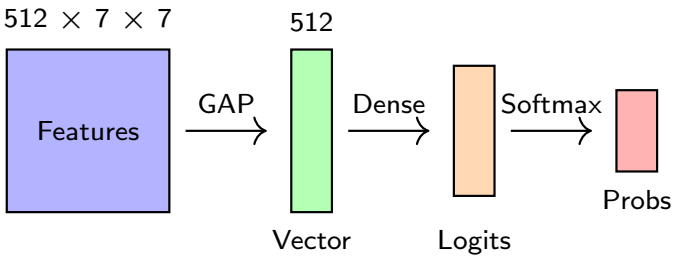
How do we connect the 3D feature maps to the final Class probabilities?

1. Global Average Pooling (GAP)
Goal: Remove spatial info to prevent overfitting.
We average the 7×7 grid for each of the 512 channels independently.

$$512 \times 7 \times 7 \xrightarrow{Mean} 512 \times 1 \times 1$$

2. Linear Layer (The Class Scores)
Goal: Combine features to make a decision.
We take a weighted sum of the 512 features for each class (like running 10 linear regressions). The outputs are **logits**: raw confidence scores.

$$512 \xrightarrow{Weights} 10 \text{ logits}$$



3. Softmax (The Probabilities)
Goal: Convert scores to percentages.
We normalize the logits so they sum to 1.

$$P(\text{Class}_i) = \frac{e^{\text{logit}_i}}{\sum_j e^{\text{logit}_j}}$$

End-to-End Walkthrough

1. **Input:** Image of a Dog ($224 \times 224 \times 3$).
2. **Low-Level Features:** Early layers detect edges and fur textures.
3. **Mid-Level Features:** Middle layers detect eyes, ears, tails.
4. **High-Level Features:** Deep layers detect "Face pattern" and "Leg pattern".
5. **Pooling:** Removes position info ("Has ears" is more important than "Ears at x=50").
6. **Classifier:** The final vector says: "High confidence in Dog features, Low confidence in Airplane features."
7. **Output:** Dog: 0.95, Cat: 0.04, Car: 0.01.